

Advanced ASIC Design

Exercise no 4: Fusion Compiler — synthesis and top-down
implementation

Krzysztof Dziembała

2025-06-11

Contents

2	Library creation	3
2.1	lab/task2/scripts/task2.tcl	3
3	MCMM and different optimisation options	5
3.1	Scripts and constraints	8
3.1.1	lab/setup/mcmm_setup.tcl	8
3.1.2	src/sdc/dut_toplevel.sdc	9
3.1.3	src/sdc/dut_toplevel_common.sdc	9
3.1.4	src/sdc/dut_toplevel_normal.sdc	9
3.1.5	src/sdc/dut_toplevel_power_save.sdc	9
3.1.6	lab/task3/scripts/task3_base.tcl	10
3.1.7	lab/task3/scripts/task3_general.tcl	11
3.1.8	lab/task3/scripts/task3_timing.tcl	11
3.1.9	lab/task3/scripts/task3_area.tcl	11
3.1.10	lab/task3/scripts/task3_power.tcl	11
4	Floorplanning	13
4.1	Automatic floorplan	13
4.2	Manual floorplan	13
4.3	Floorplan with power networks	18
4.4	Comparison	18
4.5	lab/task4/scripts/task4.tcl	19

List of Figures

3.1	Layout after <code>logic_opto</code> when optimising for area, timing or nothing.	6
3.2	Layout after <code>logic_opto</code> when optimising for power.	7
4.1	Automatic floorplan with no changes.	14
4.2	Automatic floorplan with constraints.	15
4.3	Automatic floorplan with block pin constraints.	16
4.4	Automatic floorplan with individual pin constraints on <code>clk</code>	17
4.5	Manual floorplan with tap cells.	18
4.6	Floorplan with power networks.	20

Task 2. Library creation

After the library is compiled `report_ref_libs` prints the following report:

```
135 report_ref_libs
136 *****
137 Report : Reference Library Report
138 Library: dut_toplevel.dlib
139 Version: V-2023.12
140 Date   : Wed Jun  4 13:40:08 2025
141 *****
142
143      Name                      Path
144      ↪ Location
145      -----
146      ↪ -----
147 *+ saed14rvt_ff0p88v125c      /home/student/Documents/kdz/ASIC/labs/Lab_4/
148   ↪ asic_lab4/lab/task2/work/CLIBs/saed14rvt_c.ndm
149   ↪ /home/student/Documents/kdz/ASIC/labs/Lab_4/asic_lab4/lab/task2/work/CLIBs/
150   ↪ saed14rvt_c.ndm
151 *+ saed14rvt_c_physical_only /home/student/Documents/kdz/ASIC/labs/Lab_4/
152   ↪ asic_lab4/lab/task2/work/CLIBs/saed14rvt_c_physical_only.ndm
153   ↪ /home/student/Documents/kdz/ASIC/labs/Lab_4/asic_lab4/lab/task2/work/CLIBs/
154   ↪ saed14rvt_c_physical_only.ndm
155 "*" = Library currently open
156 "+" = Library has technology information
157 1
```

2.1 lab/task2/scripts/task2.tcl

```
1 #####
2 #####                               Task 2                               #####
3 #####
4
5 ##### Sourcing main setup script
6 source -echo ../../setup/main_setup.tcl
7
8 ##### Sourcing project specyfic setup script
9 source -echo ${SetupDir}/design_setup.tcl
10
11 # Create and compile library
12 set_app_var link_library "${DB_FF} ${DB_TT} ${DB_SS}"
13 create_lib ${ResultsDir}/${DesignLibrary} -technology ${TechFile} -ref_libs
14   ↪ ${RefLib}
15 report_ref_libs
16
17 # Analyze and elaborate
18 analyze -format sverilog [glob ${SystemVerilogDir}/*.svh,sv]
19 elaborate ${DesignName}
20 set_top_module ${DesignName}
```

```
20
21 # Save as rtl_read and close
22 save_block -as ${DesignName}/rtl_read
23 list_blocks
24 close_blocks -force
25
26 # Save lib and close
27 save_lib
28 close_lib
29
30 copy_lib -from_lib ${ResultsDir}/${DesignLibrary} -to_lib
    ↪ "${SolutionDir}/${DesignLibrary}"
31
32 exit
```

Task 3. MCMM and different optimisation options

Table 3.1: Comparison of select reported parameters using different optimisation criteria. Timing is provided for scenario Normal_Slow (setup, total slack and critical length) and Normal_Fast (hold). Power data uses N/P to denote mode and S/T/F for corners.

Parameter	general/area/tim- ing	power
NS Total power (pW)	4.17×10^7	4.18×10^7
- Dynamic power (pW)	4.17×10^7	4.17×10^7
- Leakage power (pW)	1.59×10^4	1.39×10^4
NT Total power (pW)	5.99×10^7	6.01×10^7
- Dynamic power (pW)	5.96×10^7	5.98×10^7
- Leakage power (pW)	2.87×10^5	2.75×10^5
NF Total/Dynamic power (pW)	1.17×10^8	1.17×10^8
PS Total power (pW)	8.26×10^6	8.27×10^6
- Dynamic power (pW)	8.24×10^6	8.26×10^6
- Leakage power (pW)	1.59×10^4	1.39×10^4
PT Total power (pW)	1.43×10^7	1.43×10^7
- Dynamic power (pW)	1.40×10^7	1.40×10^7
- Leakage power (pW)	2.87×10^5	2.75×10^5
PF Total/Dynamic power (pW)	4.02×10^7	4.03×10^7
Critical path length (ns)	1.79	1.78
Number of violating paths	4	4
Total negative slack (ns)	-0.21	-0.21
Worst setup slack (ns)	-0.09	-0.08
Worst hold slack (ns)	0.02	0.02
Chip area	1270.506	1297.368
Total cell area	904.25	909.42
- Combinational area	473.13	478.30
- Noncombinational area	431.12	431.12
Number of nets	1605	1656
Number of cells	1174	1243

Optimizations for area (`compile.flow.high_effort_area true`) and timing (`compile.flow.high_effort_timing 2`) compiled to `logic_opto` stage are

identical to the version with those options left at default values (`false` and `0` respectively).

Setting `compile.flow.enable_power` `true` to enable power optimisations causes the compilation to produce different results. The design becomes bigger (greater area, more cells and nets), uses a little bit more power and has a tiny bit shorter critical path (by `0.01 ns`). Changes on layout are also visible (compare Fig. 3.1 and Fig. 3.2).

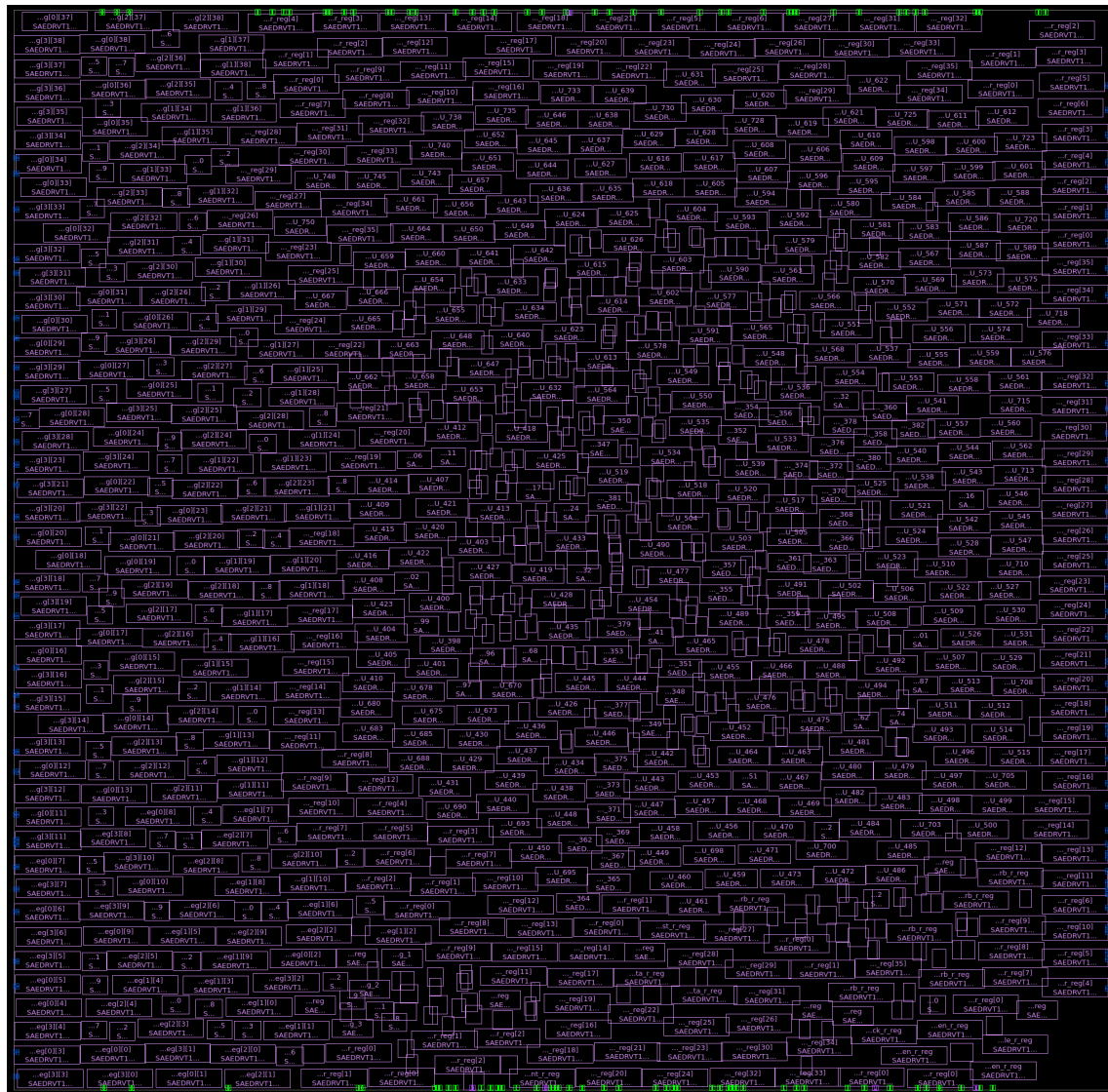


Figure 3.1: Layout after `logic_opto` when optimising for area, timing or nothing.

If full compilation is done, runs with different options set, start to show differences. Those are, however, rather small and counterintuitive — optimising for any parameter makes it worse (e.g. only the run with `compile.flow.high_effort_timing` 2 produced a single setup slack violation).

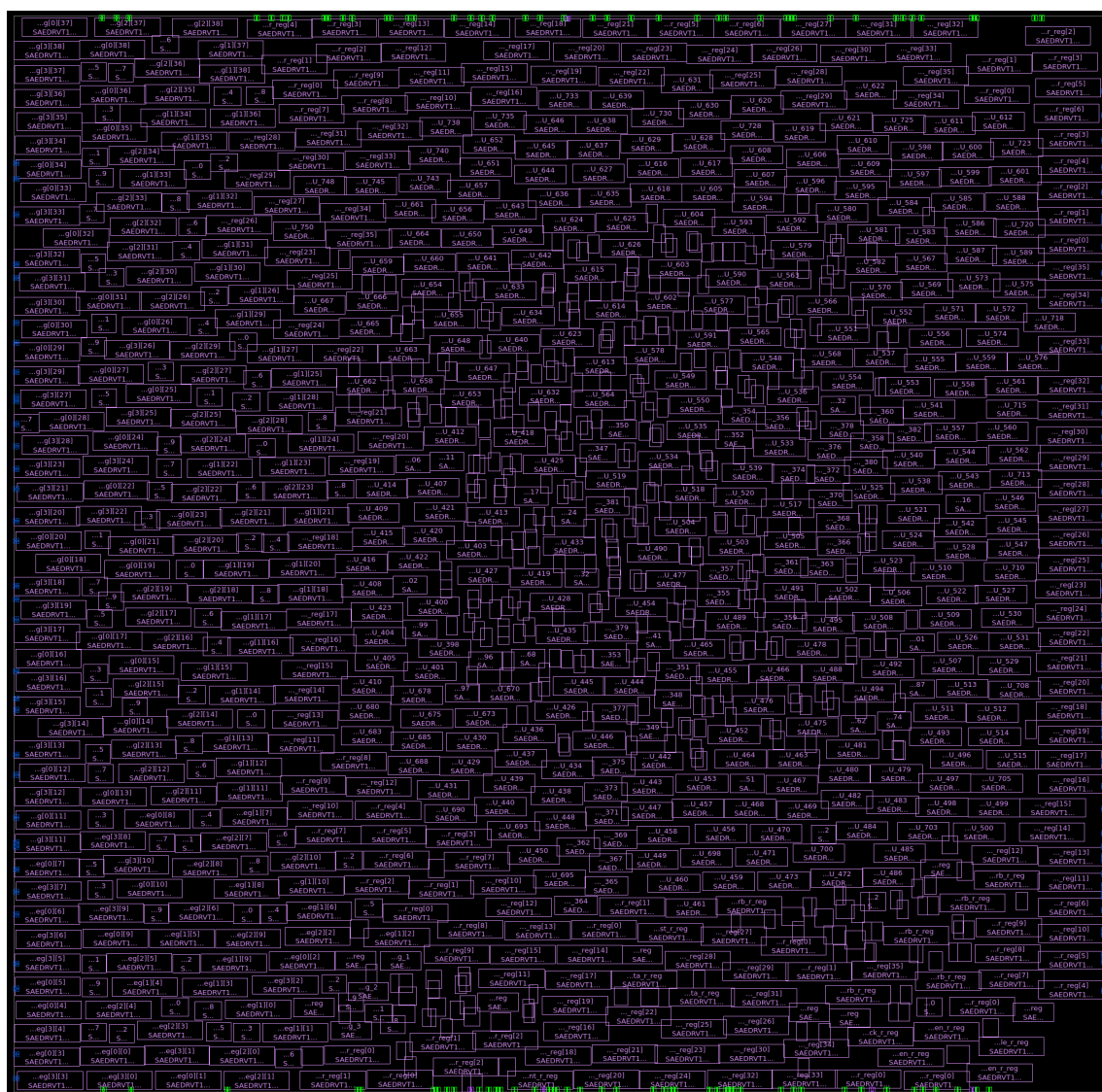


Figure 3.2: Layout after `logic_opto` when optimising for power.

3.1 Scripts and constraints

3.1.1 lab/setup/mcmm_setup.tcl

```
1 #####
2 #####          MCMM Setup Script          #####
3 #####
4
5 # Remove all MCMM related info
6 remove_corners -all
7 remove_modes -all
8 remove_scenarios -all
9
10 # Create Corner(s)
11 create_corner Fast
12 create_corner Typical
13 create_corner Slow
14
15 # Set parasitics parameters
16 set_parasitic_parameters -early_spec minTLU -late_spec minTLU -corners {Fast}
17 set_parasitic_parameters -early_spec nomTLU -late_spec nomTLU -corners {Typical}
18 set_parasitic_parameters -early_spec maxTLU -late_spec maxTLU -corners {Slow}
19
20 # Create Mode
21 foreach mode_name {Normal PowerSave} {
22     echo "Current mode: ${mode_name}"
23
24     # Create mode
25     create_mode ${mode_name}
26     current_mode ${mode_name}
27
28     # Create scenarios for current mode
29     foreach corner_name {Fast Typical Slow} {
30         create_scenario -mode ${mode_name} -corner ${corner_name} -name
↵    ${mode_name}_${corner_name}
31     }
32
33     # Read constraints for each scenario
34     set scenarios [get_attribute [get_scenarios -modes ${mode_name}] name]
35
36     echo "Current mode scenarios: ${scenarios}"
37
38     # create_clock applies to current mode and should not be redefined for each
↵    scenario - only its constraints
39     # Just ignore those warnings
40     foreach scenario ${scenarios} {
41         echo "Current scenario: ${scenario}"
42         current_scenario ${scenario}
43         read_sdc [set ${mode_name}ModeSdcFile]
44     }
45
46     # Set operating conditions for each corner and scenario
47     current_corner Fast
48     current_scenario ${mode_name}_Fast
49     set_operating_conditions ${PVT_FF}
50
```

```

51     current_corner Typical
52     current_scenario ${mode_name}_Typical
53     set_operating_conditions ${PVT_TT}
54
55     current_corner Slow
56     current_scenario ${mode_name}_Slow
57     set_operating_conditions ${PVT_SS}
58
59     # Scenario configuration
60     set_scenario_status ${mode_name}_Fast -setup false -hold true -leakage_power
↪ false -dynamic_power true -max_transition false -max_capacitance true -active
↪ true
61     set_scenario_status ${mode_name}_Typical -all -active true
62     set_scenario_status ${mode_name}_Slow -setup true -hold false -leakage_power
↪ true -dynamic_power true -max_transition true -max_capacitance false -active
↪ true
63 }

```

3.1.2 src/sdc/dut_toplevel.sdc

This file has been copied from previous laboratories.

```

1  set PERIOD 3
2  set CLOCK_NAME clk
3
4  create_clock -period $PERIOD -name $CLOCK_NAME [get_ports $CLOCK_NAME]

```

3.1.3 src/sdc/dut_toplevel_common.sdc

This file is sourced by *normal* and *power_save* constraint files, after setting up parameters.

```

1  create_clock -period $Period -name $ClockName [get_ports $ClockName]
2  set_clock_uncertainty -setup $Uncertainty [get_clocks $ClockName]
3  set_clock_transition -rise $RiseTransition [get_clocks $ClockName]
4  set_clock_transition -fall $FallTransition [get_clocks $ClockName]

```

3.1.4 src/sdc/dut_toplevel_normal.sdc

```

1  set ClockName clk
2
3  set Period 2
4  set RiseTransition 0.1
5  set FallTransition 0.2
6  set Uncertainty 0.3
7
8  source [file join [file dirname [info script]] "dut_toplevel_common.sdc"]

```

3.1.5 src/sdc/dut_toplevel_power_save.sdc

```

1  set ClockName clk
2
3  set Period 10
4  set RiseTransition 0.3

```

```

5 set FallTransition 0.5
6 set Uncertainty 0.7
7
8 source [file join [file dirname [info script]] "dut_toplevel_common.sdc"]

```

3.1.6 lab/task3/scripts/task3_base.tcl

This script is sourced by *general*, *timing*, *area* and *power* scripts which set optimisation parameters.

```

1 #####
2 ##### Task 3 #####
3 #####
4
5 ##### Sourcing main setup script
6 source -echo ../../setup/main_setup.tcl
7
8 ##### Sourcing project specyfifc setup script
9 source -echo ${SetupDir}/design_setup.tcl
10
11 # Open the design library
12 open_lib ${ResultsDir}/${DesignLibrary}
13
14 set full_compile ""
15 if { [info exists ::env(FULL_RUN)] && $::env(FULL_RUN) eq "1" } {
16     set full_compile "full_"
17     puts {Script will perform full compilation}
18 }
19
20 # Copy and open block
21 copy_block -from ${DesignName}/rtl_read -to
22     ↪ ${DesignName}/mcm_and_logic_opto_${full_compile}${opt_type}
23 open_block ${DesignName}/mcm_and_logic_opto_${full_compile}${opt_type}
24
25 # Source tech setup script
26 source -echo ${SetupDir}/technology_setup.tcl
27
28 # Read the constraints
29 read_sdc -echo ${SdcFile}
30
31 # MCMM setup
32 source -echo ${SetupDir}/mcm_setup.tcl
33
34 if {[info exists opt_setting]} {
35     set_app_options -name compile.flow.${opt_setting} -value ${opt_value}
36 }
37 report_app_options -non_default
38
39 if { $full_compile eq "" } {
40     # Initial mapping
41     compile_fusion -to logic_opto
42 } else {
43     compile_fusion
44 }
45
46 # Collecting the reports

```

```

46 set TargetName "logic_opto_${full_compile}${opt_type}"
47 generateReports ${TargetName}
48
49 # Gate level netlist generation
50 write_verilog ${ResultsDir}/${DesignName}_${full_compile}${TargetName}.v
51
52 # Save layout image
53 # gui_start
54 # set window [gui_get_current_window -mru -type Layout]
55 gui_start
56 set top [gui_create_window -type TopLevel]
57 set layout [gui_create_window -type Layout -parent $top]
58 gui_show_window -window $top -show_state {maximized}
59 gui_show_window -window $layout -show_state {maximized}
60 gui_write_window_image -window $layout -clip -file
    ↪ ${ScreenshotsDir}/layout_${full_compile}${TargetName}.png
61 gui_stop
62
63 # Saving block and library
64 current_block
65 get_blocks -all
66 list_blocks
67 save_block
68 save_lib
69 close_blocks
70 close_lib
71
72 exit
73

```

3.1.7 lab/task3/scripts/task3_general.tcl

```

1 set opt_type "general"
2 source -echo ../scripts/task3_base.tcl

```

3.1.8 lab/task3/scripts/task3_timing.tcl

```

1 set opt_type "timing"
2 set opt_setting "high_effort_timing"
3 set opt_value 2
4 source -echo ../scripts/task3_base.tcl

```

3.1.9 lab/task3/scripts/task3_area.tcl

```

1 set opt_type "area"
2 set opt_setting "high_effort_area"
3 set opt_value true
4 source -echo ../scripts/task3_base.tcl

```

3.1.10 lab/task3/scripts/task3_power.tcl

```

1 set opt_type "power"
2 set opt_setting "enable_power"
3 set opt_value true

```

```
4 source -echo ../scripts/task3_base.tcl
```

Task 4. Floorplanning

4.1 Automatic floorplan

Running `report_app_options -non_default` after `app_options -name place.coarse.continue_on_missing_scandef -value true` and `set_app_options -name compile.auto_floorplan.enable -value true` reports only the former option being changed from default. `report_app_options compile.auto*` lists all options starting with `compile.auto` and among `compile.auto_floorplan.*` options there is none with the name `enable`. It looks like it is enabled by default with no option to disable, other than initializing floorplan manually.

It is important to run `compile_fusion`, at least to `logic_opto` after setting any constraints, or changes will not be visible.

The images below present how the floorplan changes as constraints are added:

1. The floorplan is square-looking (Fig. 4.1)
2. Constraints change the shape to a rectangle that stretches vertically, has some free space and a lot of free space around the core (Fig. 4.2)
3. Pins are spread apart and present on top and both sides, causing the core content to be flipped closer to new pin locations (Fig. 4.3)
4. A single (clk) pin appears on the bottom edge of the chip (Fig. 4.4)

Finally, `report_congestion -rerun_global_router` reported the following small number of congestions:

Report : congestion

Design : dut_toplevel

Version: V-2023.12

Date : Wed Jun 4 15:46:29 2025

Layer	overflow		# GRCs has	
Name	total	max	overflow (%)	max overflow
Both Dirs	52	1	52 (0.22%)	52
H routing	7	1	7 (0.06%)	7
V routing	45	1	45 (0.38%)	45

4.2 Manual floorplan

It is important to close the library and open again, if `initialize_floorplan` is to be used after previous compilation that used automatic floorplan, or, for some reason, automatic floorplan will still be used.

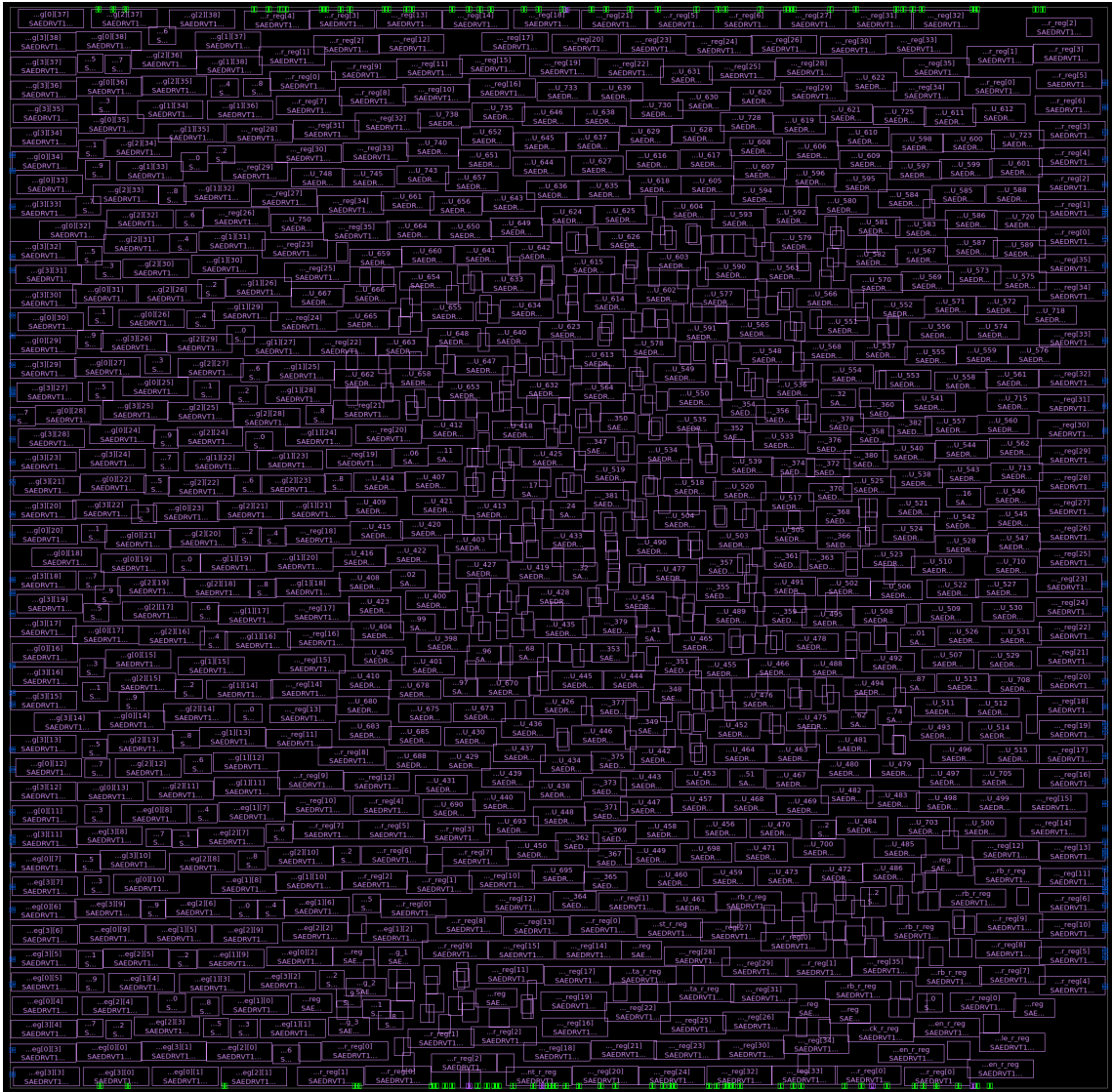


Figure 4.1: Automatic floorplan with no changes.

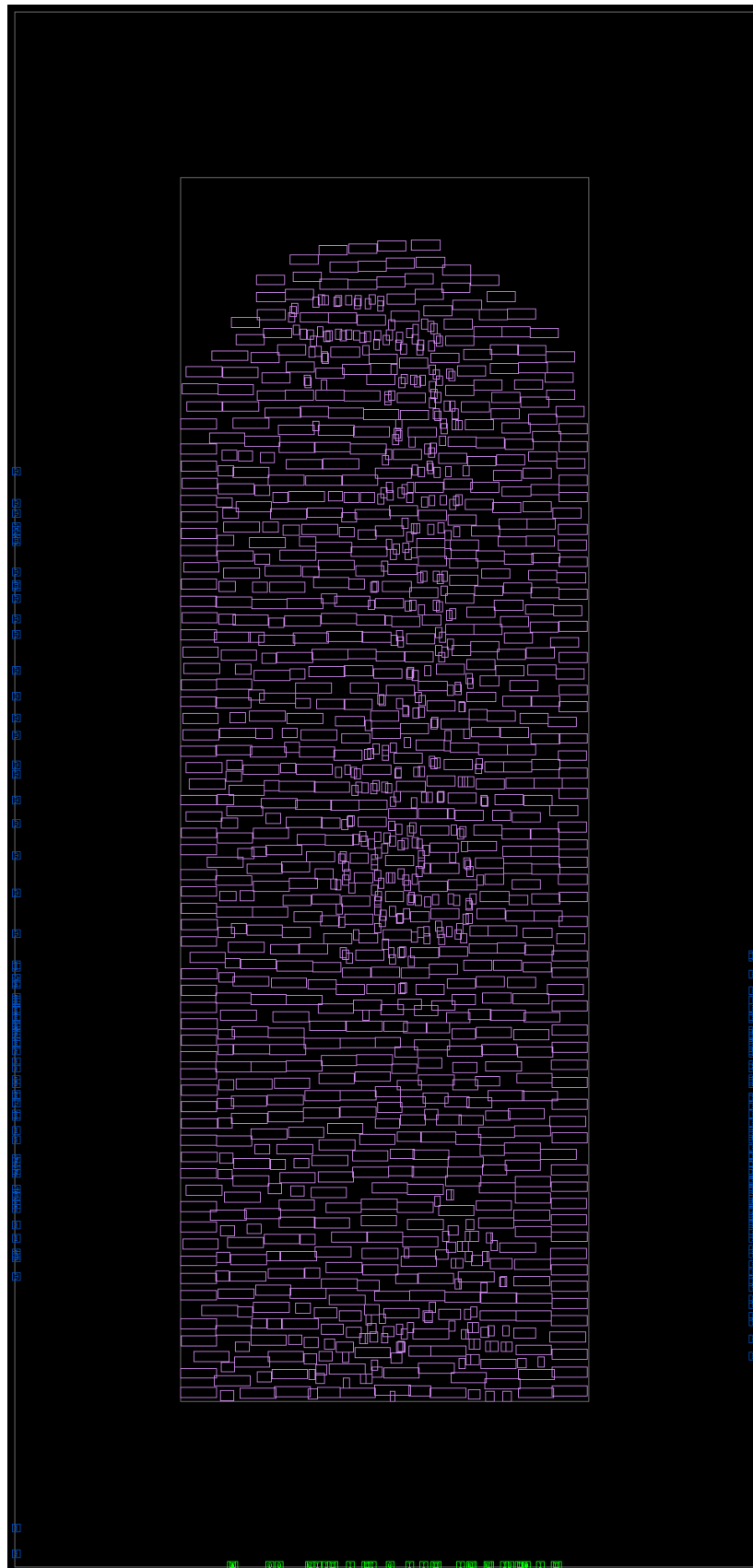


Figure 4.2: Automatic floorplan with constraints.

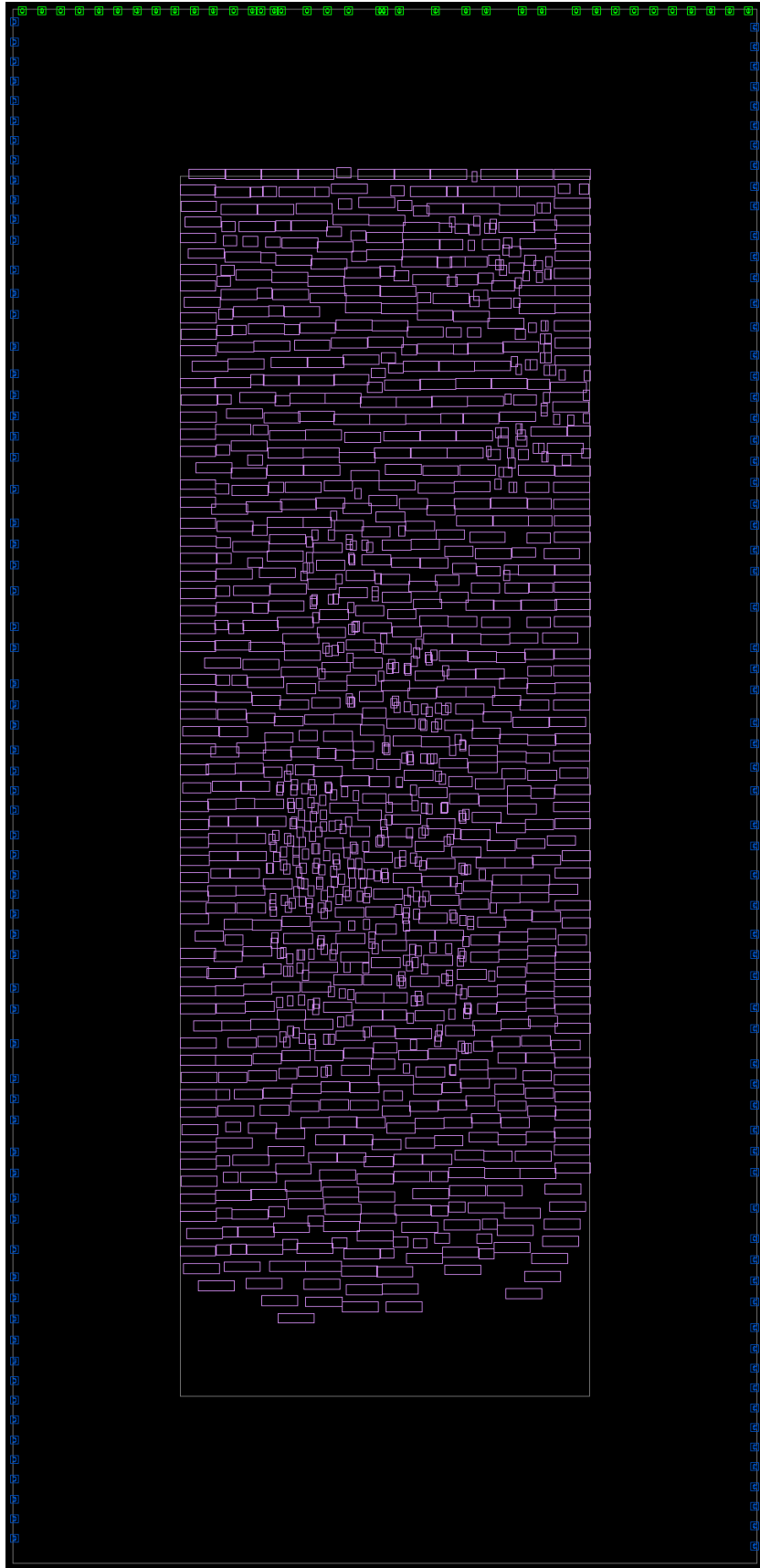


Figure 4.3: Automatic floorplan with block pin constraints.

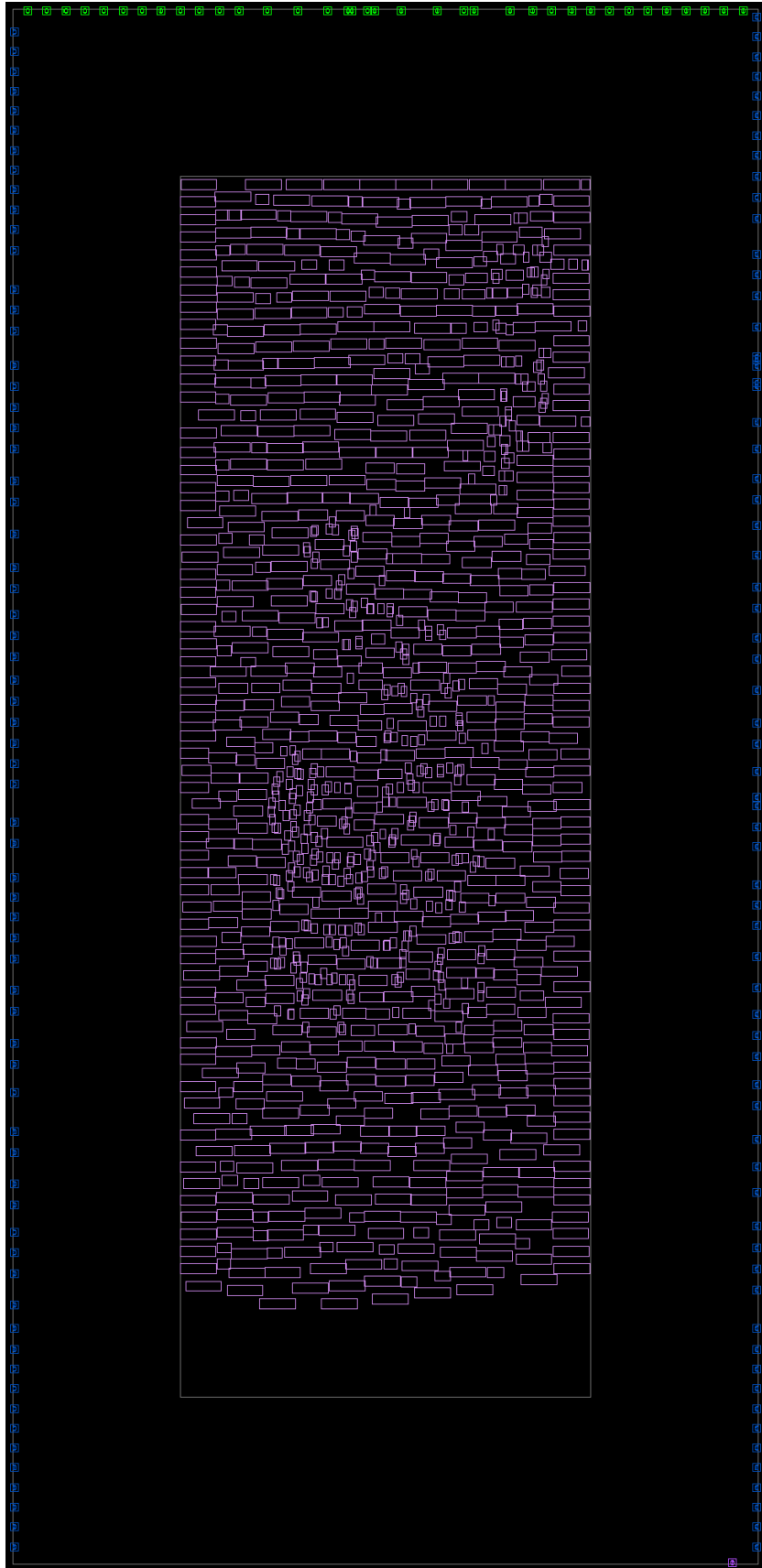


Figure 4.4: Automatic floorplan with individual pin constraints on clk.

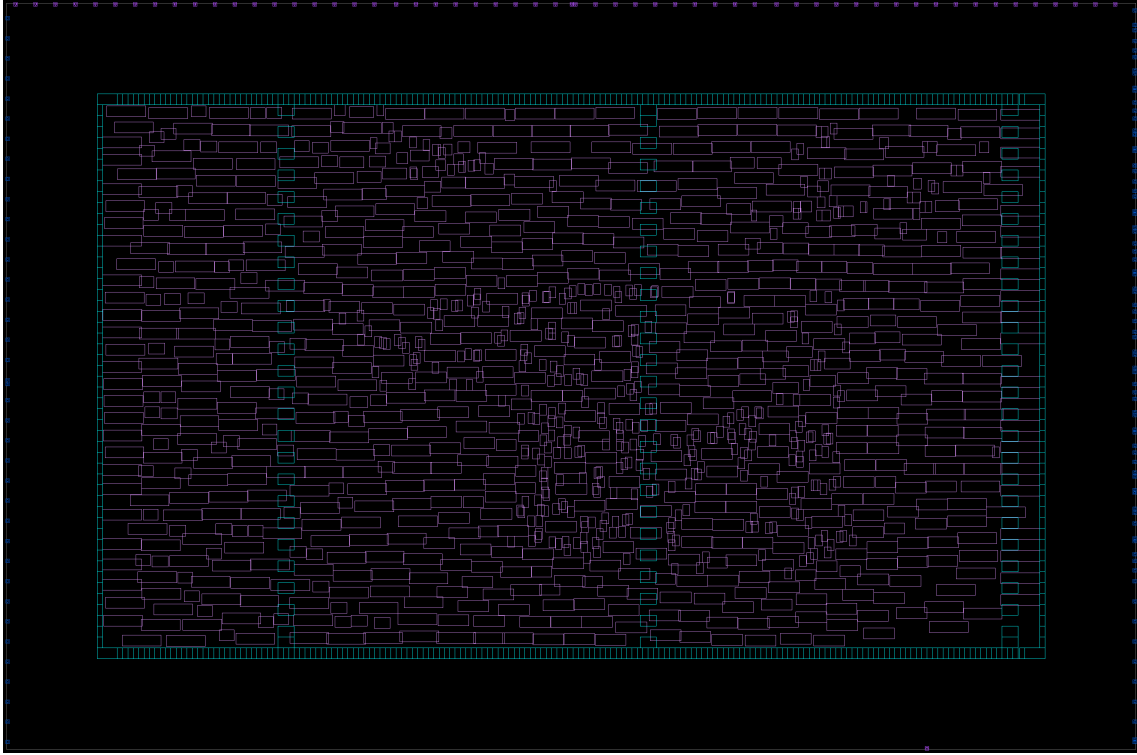


Figure 4.5: Manual floorplan with tap cells.

The manual floorplan is a horizontal rectangle that is stretched less than the automatic one. Like in the automatic floorplan, pins are allowed on left, top and right sides, with only clk on the bottom. This floorplan features tap cells in three columns inside and boundary/end cap cells forming a ring around the core.

4.3 Floorplan with power networks

The last floorplan version prepared in this exercise is based on **manual floorplan** with added power networks.

4.4 Comparison

Table 4.1: Comparison of select reported parameters using different floorplans. Timing is provided for scenario Normal_Slow (setup, total slack and critical length) and Normal_Fast (hold). Power data uses N/P to denote mode and S/T/F for corners.

Parameter	tuned automatic	manual	manual with power
NS Total power (pW)	4.38×10^7	4.16×10^7	4.23×10^7

Parameter	tuned automatic	manual	manual with power
- Dynamic power (pW)	4.38×10^7	4.16×10^7	4.23×10^7
- Leakage power (pW)	1.60×10^4	1.59×10^4	1.60×10^4
NT Total power (pW)	6.24×10^7	5.97×10^7	6.06×10^7
- Dynamic power (pW)	6.21×10^7	5.94×10^7	6.03×10^7
- Leakage power (pW)	2.88×10^5	2.87×10^5	2.88×10^5
NF Total/Dynamic power (pW)	1.20×10^8	1.17×10^8	1.18×10^8
PS Total power (pW)	8.68×10^6	8.23×10^6	8.38×10^6
- Dynamic power (pW)	8.67×10^6	8.22×10^6	8.36×10^6
- Leakage power (pW)	1.60×10^4	1.59×10^4	1.60×10^4
PT Total power (pW)	1.48×10^7	1.42×10^7	1.44×10^7
- Dynamic power (pW)	1.45×10^7	1.39×10^7	1.41×10^7
- Leakage power (pW)	2.88×10^5	2.87×10^5	2.88×10^5
PF Total/Dynamic power (pW)	4.08×10^7	4.02×10^7	4.04×10^7
Critical path length (ns)	1.80	1.74	1.82
Number of violating paths	5	3	5
Total negative slack (ns)	-0.30	-0.07	-0.38
Worst setup slack (ns)	-0.11	-0.04	-0.13
Worst hold slack (ns)	0.02	0.02	0.02
Chip area	4153.738	2567.502	2567.502
Core area	1798.378	1632.322	1632.322
Total cell area	905.45	904.92	905.01
- Combinational area	474.33	473.79	473.79
- Noncombinational area	431.12	431.12	431.21
Number of nets	1602	1602	1607
Number of cells	1171	1171	1174

According to the data from reports presented in Table 4.1, the constraints used in automatic floorplan make it disadvantaged with regards to power, timing and area compared to the manual less stretched floorplan. Core area of the stretched automatic floorplan is over 10% higher than this of manual, with chip area over 61% greater. Chip area is an important factor, because it dictates how many chips can fit on a single wafer, affects yield rates and finally this translates to costs.

Adding power network results in timing and power parameters significantly worsening. It is possible that it's just a result of `compile_fusion` randomness, but this is only my speculation.

4.5 lab/task4/scripts/task4.tcl

```

1 #####
2 #####                                Task 4                                #####

```

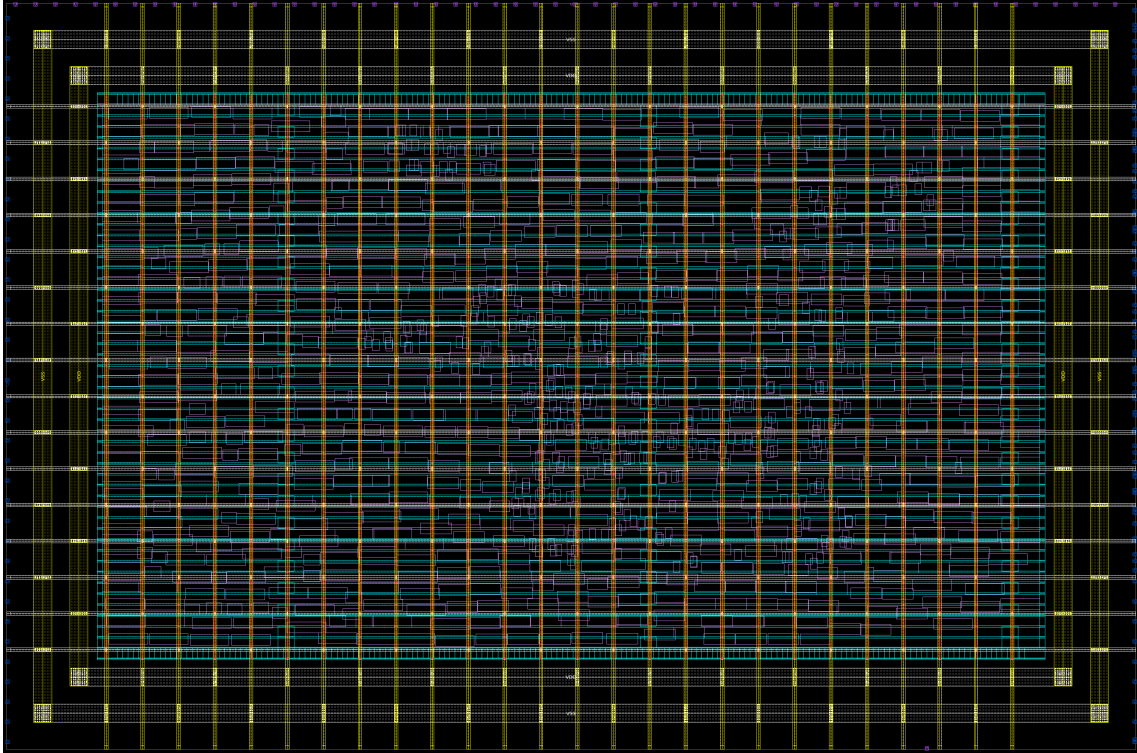


Figure 4.6: Floorplan with power networks.

```

3 #####
4
5 ##### Sourcing main setup script
6 source -echo ../../setup/main_setup.tcl
7
8 ##### Sourcing project specyfic setup script
9 source -echo ${SetupDir}/design_setup.tcl
10
11 open_lib ${ResultsDir}/${DesignLibrary}
12
13 proc make_layout_screenshot {suffix} {
14     variable ScreenshotsDir
15
16     gui_start
17     set top [gui_create_window -type TopLevel]
18     set layout [gui_create_window -type Layout -parent $top]
19     gui_show_window -window $top -show_state {maximized}
20     gui_show_window -window $layout -show_state {maximized}
21     gui_write_window_image -window $layout -clip -file
22     ↪ ${ScreenshotsDir}/floorplan_layout_${suffix}.png
23     gui_stop
24 }
25 #####
26 ##### Auto Floorplan #####
27 #####
28 copy_block -from ${DesignName}/rtl_read -to ${DesignName}/auto_floorplan
29 open_block ${DesignName}/auto_floorplan
30

```

```

31 source -echo ${SetupDir}/technology_setup.tcl
32 read_sdc -echo ${SdcFile}
33 source -echo ${SetupDir}/mcm_setup.tcl
34
35 set_app_options -name place.coarse.continue_on_missing_scandef -value true
36 set_app_options -name compile.auto_floorplan.enable -value true
37 report_app_options -non_default
38
39 compile_fusion -check_only
40 compile_fusion -to logic_opto
41
42 make_layout_screenshot automatic_default
43
44 # Floorplan constraints
45 set_auto_floorplan_constraints \
46     -control_type core \
47     -core_utilization 0.5 \
48     -core_offset 10 \
49     -shape R \
50     -side_ratio {1 3} \
51     -flip_first_row true
52 report_auto_floorplan_constraints
53
54 make_layout_screenshot automatic_constraints
55 compile_fusion -to logic_opto
56 make_layout_screenshot automatic_constraints_compiled
57
58 # Block pin constraints
59 set_block_pin_constraints -self \
60     -allowed_layers {M2 M3} \
61     -sides {1 2 3} \
62     -pin_spacing_distance 1 \
63     -width 0.19 \
64     -length 0.19
65 report_block_pin_constraints -self
66
67 make_layout_screenshot automatic_block_pin_constraints
68 compile_fusion -to logic_opto
69 make_layout_screenshot automatic_block_pin_constraints_compiled
70
71 # Individual pin constraints
72 set_individual_pin_constraints -ports [get_port clk] -sides 4 -allowed_layers M4
73 report_individual_pin_constraints
74
75 make_layout_screenshot automatic_individual_pin_constraints
76 compile_fusion -to logic_opto
77 make_layout_screenshot automatic_individual_pin_constraints_compiled
78
79 # Final reports
80 report_congestion -rerun_global_router
81 generateReports tuned_auto_floorplan
82
83 # Save results
84 write_floorplan -output ${ResultsDir}/auto_floorplan_files -exclude {cells nets}
85 write_def -exclude {cells nets} ${ResultsDir}/auto_floorplan.def
86
87 save_block

```

```

88 close_blocks
89 reset_app_options {compile.auto_floorplan.enable}
90 save_lib
91 # Close library - otherwise it'll use previous floorplan
92 close_lib
93
94 #####
95 ##### Manual Floorplan #####
96 #####
97 open_lib ${ResultsDir}/${DesignLibrary}
98 copy_block -from ${DesignName}/auto_floorplan -to ${DesignName}/manual_floorplan
99 open_block ${DesignName}/manual_floorplan
100 report_app_options -non_default
101
102 # Initialize the floorplan
103 initialize_floorplan \
104     -control_type core \
105     -core_utilization 0.55 \
106     -core_offset 5 \
107     -shape R \
108     -side_ratio {2.5 1.5} \
109     -flip_first_row true
110
111 # Set pin placement constraints
112 set ports [remove_from_collection [get_ports] {VDD VSS}]
113
114 set_block_pin_constraints -self \
115     -allowed_layers {M3 M4} \
116     -sides {1 2 3} \
117     -pin_spacing_distance 1 \
118     -width 0.11 \
119     -length 0.11
120 set_individual_pin_constraints -ports [get_port clk] -sides 4 -allowed_layers M5
121
122 place_pins -self -ports ${ports}
123
124 # Insert boundary and TAP cells in the design
125 source -echo ../scripts/insert_special_physical_cells.tcl
126
127 make_layout_screenshot manual
128 compile_fusion -to logic_opto
129 make_layout_screenshot manual_compiled
130
131 # Write out the created floorplan
132 write_floorplan -output ${ResultsDir}/manual_floorplan_files -exclude {cells
    ↪ nets}
133
134 # Write DEF for created floorplan
135 write_def -exclude {cells nets} ${ResultsDir}/manual_floorplan.def
136
137 generateReports manual_floorplan
138
139 get_blocks -all
140 save_block
141 save_lib
142
143 close_blocks

```

```

144 # Close library - otherwise it'll use previous floorplan
145 close_lib
146
147 #####
148 #####      Creating power and ground network      #####
149 #####
150 open_lib ${ResultsDir}/${DesignLibrary}
151 copy_block -from ${DesignName}/rtl_read -to ${DesignName}/final_floorplan
152 open_block ${DesignName}/final_floorplan
153
154 # Source tech setup script
155 source -echo ${SetupDir}/technology_setup.tcl
156
157 # Read the constraints
158 read_sdc -echo ${SdcFile}
159
160 # MCMM setup
161 source -echo ${SetupDir}/mcm_setup.tcl
162
163 # Read floorplan from DEF
164 read_def ${ResultsDir}/manual_floorplan.def
165
166 # Insert boundary and TAP cells in the design
167 source -echo ../scripts/insert_special_physical_cells.tcl
168
169 # Create power/ground network
170 source -echo ../scripts/create_pg_network.tcl
171
172 make_layout_screenshot final
173 compile_fusion -to logic_opto
174 make_layout_screenshot final_compiled
175
176 generateReports final_floorplan
177
178 save_block
179 save_lib
180 close_blocks
181 close_lib
182
183 exit
184

```